```c
/* ============================================================
*  CAN SNIFFER z OLED — Blue Pill STM32F103C8T6
*  Wyświetla ostatnie 5 ramek CAN na ekranie SSD1306 128x64
*  Wszystkie ramki są akceptowane (filtr otwarty)
* ============================================================ */
#include "main.h"
#include "ssd1306.h"
#include "ssd1306_fonts.h"
#include <string.h>
#include <stdio.h>
/* ============================================================
*  Handles HAL
* ============================================================ */
CAN_HandleTypeDef  hcan;
UART_HandleTypeDef huart2;
I2C_HandleTypeDef  hi2c1;
/* ============================================================
*  Zmienne globalne
* ============================================================ */
char uart_buf[80];
#define SNIFFER_LINES 3
char          oled_lines[SNIFFER_LINES][20];
volatile uint8_t oled_dirty = 0;
volatile uint32_t frame_count = 0;
/* ============================================================
*  Prototypy
* ============================================================ */
void SystemClock_Config(void);
static void MX_GPIO_Init(void);
static void MX_USART2_UART_Init(void);
static void MX_CAN_Init(void);
static void MX_I2C1_Init(void);
void uart_print(const char *msg);
/* ============================================================
*  sniffer_push
* ============================================================ */
static void sniffer_push(const char *txt)
{
  for (int i = SNIFFER_LINES - 1; i > 0; i--)
     memcpy(oled_lines[i], oled_lines[i - 1], sizeof(oled_lines[0]));
  strncpy(oled_lines[0], txt, sizeof(oled_lines[0]) - 1);
  oled_lines[0][sizeof(oled_lines[0]) - 1] = '\0';
  oled_dirty = 1;
}
/* ============================================================
*  oled_redraw — wywołuj TYLKO z main loop, nigdy z IRQ
* ============================================================ */
static void oled_redraw(void)
{
  char header[20];
  snprintf(header, sizeof(header), "SNIFFER #%lu", frame_count);
  ssd1306_Fill(Black);
```

```
  ssd1306_SetCursor(0, 0);
  ssd1306_WriteString(header, Font_7x10, White);
  for (int i = 0; i < SNIFFER_LINES; i++)
  {
    ssd1306_SetCursor(0, 12 + i * 11);
    if (oled_lines[i][0] != '\0')
      ssd1306_WriteString(oled_lines[i], Font_7x10, White);
    else
      ssd1306_WriteString("---", Font_7x10, White);
  }
  ssd1306_UpdateScreen();
}
/* ============================================================
*  CALLBACK CAN RX
* ============================================================ */
void HAL_CAN_RxFifo0MsgPendingCallback(CAN_HandleTypeDef *hcan_h)
{
  CAN_RxHeaderTypeDef hdr;
  uint8_t data[8];
  if (HAL_CAN_GetRxMessage(hcan_h, CAN_RX_FIFO0, &hdr, data) != HAL_OK)
    return;
  frame_count++;
  char line[20];
  if (hdr.DLC == 1)
  {
    snprintf(line, sizeof(line), "0x%03lX [%02X]",
          hdr.StdId, data[0]);
  }
  else
  {
    char bytes[10] = {0};
    int pos = 0;
    for (uint8_t i = 0; i < hdr.DLC && i < 3; i++)
      pos += snprintf(bytes + pos, sizeof(bytes) - pos,
                "%02X ", data[i]);
    snprintf(line, sizeof(line), "0x%03lX %s", hdr.StdId, bytes);
  }
  sniffer_push(line);
  /* Pełna ramka na UART */
  int pos = snprintf(uart_buf, sizeof(uart_buf),
              "ID=0x%03lX DLC=%lu [", hdr.StdId, hdr.DLC);
  for (uint8_t i = 0; i < hdr.DLC; i++)
    pos += snprintf(uart_buf + pos, sizeof(uart_buf) - pos,
                "%02X%s", data[i], (i < hdr.DLC - 1) ? " " : "");
  snprintf(uart_buf + pos, sizeof(uart_buf) - pos, "]\r\n");
  uart_print(uart_buf);
}
/* ============================================================
*  MAIN
* ============================================================ */
int main(void)
{
```

```c
  HAL_Init();
  SystemClock_Config();
  MX_GPIO_Init();
  MX_USART2_UART_Init();
  MX_CAN_Init();
  MX_I2C1_Init();
  memset(oled_lines, 0, sizeof(oled_lines));
  HAL_Delay(100);
  ssd1306_Init();
  ssd1306_Fill(Black);
  ssd1306_SetCursor(0, 0);
  ssd1306_WriteString("CAN SNIFFER", Font_7x10, White);
  ssd1306_SetCursor(0, 14);
  ssd1306_WriteString("WAITING...", Font_7x10, White);
  ssd1306_UpdateScreen();
  uart_print("=== CAN SNIFFER READY ===\r\n");
  CAN_FilterTypeDef filter = {0};
  filter.FilterBank           = 0;
  filter.FilterMode           = CAN_FILTERMODE_IDMASK;
  filter.FilterScale          = CAN_FILTERSCALE_32BIT;
  filter.FilterIdHigh         = 0x0000;
  filter.FilterIdLow          = 0x0000;
  filter.FilterMaskIdHigh     = 0x0000;
  filter.FilterMaskIdLow      = 0x0000;
  filter.FilterFIFOAssignment = CAN_RX_FIFO0;
  filter.FilterActivation     = ENABLE;
  HAL_CAN_ConfigFilter(&hcan, &filter);
  HAL_CAN_Start(&hcan);
  HAL_CAN_ActivateNotification(&hcan, CAN_IT_RX_FIFO0_MSG_PENDING);
  for (int i = 0; i < 3; i++)
  {
    HAL_GPIO_WritePin(GPIOC, GPIO_PIN_13, GPIO_PIN_RESET);
    HAL_Delay(80);
    HAL_GPIO_WritePin(GPIOC, GPIO_PIN_13, GPIO_PIN_SET);
    HAL_Delay(80);
  }
  while (1)
  {
    if (oled_dirty)
    {
      oled_dirty = 0;
      oled_redraw();
    }
  }
}
/* ============================================================
*  Init functions
* ============================================================ */
void uart_print(const char *msg)
{
  HAL_UART_Transmit(&huart2, (uint8_t*)msg, strlen(msg), 100);
}
```

```
void SystemClock_Config(void)
{
  RCC_OscInitTypeDef RCC_OscInitStruct = {0};
  RCC_ClkInitTypeDef RCC_ClkInitStruct = {0};
  RCC_OscInitStruct.OscillatorType  = RCC_OSCILLATORTYPE_HSE;
  RCC_OscInitStruct.HSEState        = RCC_HSE_ON;
  RCC_OscInitStruct.HSEPredivValue  = RCC_HSE_PREDIV_DIV1;
  RCC_OscInitStruct.HSIState        = RCC_HSI_ON;
  RCC_OscInitStruct.PLL.PLLState    = RCC_PLL_ON;
  RCC_OscInitStruct.PLL.PLLSource   = RCC_PLLSOURCE_HSE;
  RCC_OscInitStruct.PLL.PLLMUL      = RCC_PLL_MUL9;
  if (HAL_RCC_OscConfig(&RCC_OscInitStruct) != HAL_OK) Error_Handler();
  RCC_ClkInitStruct.ClockType      = RCC_CLOCKTYPE_HCLK | RCC_CLOCKTYPE_SYSCLK
                                   | RCC_CLOCKTYPE_PCLK1 | RCC_CLOCKTYPE_PCLK2;
  RCC_ClkInitStruct.SYSCLKSource   = RCC_SYSCLKSOURCE_PLLCLK;
  RCC_ClkInitStruct.AHBCLKDivider  = RCC_SYSCLK_DIV1;
  RCC_ClkInitStruct.APB1CLKDivider = RCC_HCLK_DIV2;
  RCC_ClkInitStruct.APB2CLKDivider = RCC_HCLK_DIV1;
  if (HAL_RCC_ClockConfig(&RCC_ClkInitStruct, FLASH_LATENCY_2) != HAL_OK)
Error_Handler();
}
static void MX_CAN_Init(void)
{
  hcan.Instance                  = CAN1;
  hcan.Init.Prescaler            = 4;
  hcan.Init.Mode                 = CAN_MODE_NORMAL;
  hcan.Init.SyncJumpWidth        = CAN_SJW_1TQ;
  hcan.Init.TimeSeg1             = CAN_BS1_15TQ;
  hcan.Init.TimeSeg2             = CAN_BS2_2TQ;
  hcan.Init.TimeTriggeredMode    = DISABLE;
  hcan.Init.AutoBusOff           = DISABLE;
  hcan.Init.AutoWakeUp           = DISABLE;
  hcan.Init.AutoRetransmission   = ENABLE;
  hcan.Init.ReceiveFifoLocked    = DISABLE;
  hcan.Init.TransmitFifoPriority = DISABLE;
  if (HAL_CAN_Init(&hcan) != HAL_OK) Error_Handler();
}
static void MX_USART2_UART_Init(void)
{
  huart2.Instance          = USART2;
  huart2.Init.BaudRate     = 115200;
  huart2.Init.WordLength   = UART_WORDLENGTH_8B;
  huart2.Init.StopBits     = UART_STOPBITS_1;
  huart2.Init.Parity       = UART_PARITY_NONE;
  huart2.Init.Mode         = UART_MODE_TX_RX;
  huart2.Init.HwFlowCtl    = UART_HWCONTROL_NONE;
  huart2.Init.OverSampling = UART_OVERSAMPLING_16;
  if (HAL_UART_Init(&huart2) != HAL_OK) Error_Handler();
}
static void MX_GPIO_Init(void)
{
  GPIO_InitTypeDef GPIO_InitStruct = {0};
```

```
  __HAL_RCC_GPIOC_CLK_ENABLE();
  __HAL_RCC_GPIOD_CLK_ENABLE();
  __HAL_RCC_GPIOA_CLK_ENABLE();
  __HAL_RCC_GPIOB_CLK_ENABLE();
  HAL_GPIO_WritePin(GPIOC, GPIO_PIN_13, GPIO_PIN_SET);
  GPIO_InitStruct.Pin   = GPIO_PIN_13;
  GPIO_InitStruct.Mode  = GPIO_MODE_OUTPUT_PP;
  GPIO_InitStruct.Pull  = GPIO_NOPULL;
  GPIO_InitStruct.Speed = GPIO_SPEED_FREQ_LOW;
  HAL_GPIO_Init(GPIOC, &GPIO_InitStruct);
}
static void MX_I2C1_Init(void)
{
  hi2c1.Instance             = I2C1;
  hi2c1.Init.ClockSpeed      = 100000;
  hi2c1.Init.DutyCycle       = I2C_DUTYCYCLE_2;
  hi2c1.Init.OwnAddress1     = 0;
  hi2c1.Init.AddressingMode  = I2C_ADDRESSINGMODE_7BIT;
  hi2c1.Init.DualAddressMode = I2C_DUALADDRESS_DISABLE;
  hi2c1.Init.OwnAddress2     = 0;
  hi2c1.Init.GeneralCallMode = I2C_GENERALCALL_DISABLE;
  hi2c1.Init.NoStretchMode   = I2C_NOSTRETCH_DISABLE;
  if (HAL_I2C_Init(&hi2c1) != HAL_OK) Error_Handler();
}
void Error_Handler(void)
{
  __disable_irq();
  while (1) {}
}
```